

Senior Full-Stack Developer / AI-Native Tech Lead Case Study

Objective:

This exercise aims to evaluate the candidate's ability to design and implement a realistic team-based full-stack MVP using Next.js App Router and Supabase.

We are looking for:

- AI-native engineering workflow
- Strong architectural thinking
- Clean data modeling
- Team-based ownership and access control
- Correct authentication and session handling
- Product awareness beyond raw implementation
- Clear communication and ownership

The case is designed to be completed within **3 days**.

We are not evaluating pixel-perfect UI. We care more about structure, decisions, data ownership, reasoning, and maintainability.

AI-assisted development is expected. We prefer candidates to use agentic engineering tools such as Cursor, Claude Code, Codex, Windsurf, or similar systems.

Replit, Lovable, Bolt, and similar no-code or app-generation platforms should not be used as the primary implementation approach.

A submission where most of the code is AI-generated is completely acceptable. In fact, we are especially interested in understanding how the candidate uses AI tools responsibly, reviews output, structures the workflow, and maintains quality.

Case Description:

You are working on a fictional early-stage product under Vizio Ventures.

The team is building a team-based social media platform and wants to validate a first MVP slice quickly.

You are asked to implement a clean, minimal, production-aware solution, not a fully finished product.

The goal is to show how you think when building real features under time constraints, especially around shared accounts, team ownership, privacy, messaging, and data access.

The Product

Build a team-based social media web application with authentication, team ownership, posting, team-to-team messaging, social relationships, and privacy controls.

Users should be able to:

- Sign in
- Belong to exactly one team
- Create posts on behalf of their team
- Follow other teams
- Send direct messages to other teams
- View a global feed

There are no individual profiles. All actions are performed under a team identity.

What We Expect You To Do

1. Authentication & Session Handling

Implement authentication using Supabase Auth.

Requirements:

- Email + password login
- Google OAuth login
- Persistent session handling
- Clear logged-in vs logged-out state handling

We expect:

- Clean auth flow
- Correct session persistence
- No hardcoded shortcuts
- Clear association between user and team after login

2. Team Model

This product is team-based, not user-centric.

Rules:

- Each user belongs to exactly one team
- All users in a team share the same permissions
- Teams can be either Public or Private
- Role management is intentionally out of scope

You should:

- Create or seed a team during signup, or document your approach
- Ensure all content is scoped to the team
- Keep the model simple and explicit

Focus on:

- Correct ownership
- Clean relationships
- Predictable data access

3. Posting System

Team members can create text-based posts.

Requirements:

- Any team member can create a post
- Posts belong to a team, not an individual user
- Posts include text content and created timestamp

We are looking for:

- Clean data modeling
- Correct team ownership
- Simple, modern, readable UI logic and interaction design
- Proper async and error handling

4. Follow System

Teams can follow other teams.

Rules:

- A team cannot follow itself
- Following is one-directional
- Public teams can be followed immediately
- Private teams require follow request approval before a follow relationship is established

Requirements:

- Follow / unfollow another team
- Send a follow request to a private team
- Approve or reject incoming follow requests
- Store follow relationships and follow requests in the database

Focus on:

- Data integrity
- Clear constraints
- Authorization rules
- Simple, explicit logic

5. Team Messaging

Teams can communicate directly with other teams.

Rules:

- Messages are sent between teams, not individual users
- Any member of a team can participate in conversations on behalf of their team
- A team cannot message itself

Requirements:

- Start a conversation with another team
- Send and receive messages
- Display conversation history ordered by newest message timestamp
- Persist messages in the database

Focus on:

- Clear ownership model
- Correct team scoping
- Simple and predictable data relationships

- Readable messaging UI

6. Home Feed

The home page displays a global feed.

Feed behavior:

- Shows posts from all public teams
- Shows posts from private teams only if the viewing team is an approved follower
- Ordered by newest first
- Publicly viewable without authentication for public content

We are looking for:

- Clean querying
- Predictable ordering
- Reasonable performance assumptions
- Proper handling of visibility rules

7. Data Model & Security

Design the Supabase data model, relationships, and RLS policies.

We expect:

- Users can only act on behalf of their own team
- Posts, follows, follow requests, and messages are correctly scoped
- Private team visibility is properly enforced
- Schema and RLS choices are clearly explained
- No overly complex abstractions

8. Architecture & Critical Flows

We expect the candidate to explain the system, not only implement it.

Please include in the README:

- High-level system architecture
- ERD / Entity Relationship Diagram
- Key trade-offs and alternatives
- Risks and mitigations

Production-grade diagrams are not required. Simple screenshots, Mermaid diagrams, Excalidraw sketches, or similar formats are sufficient.

9. Technical Structure & Quality

Technical expectations:

- Next.js App Router
- Supabase
- Clean async and error handling
- Reasonable folder structure
- Readable, maintainable code

You may use:

- MakerKit Next.js + Supabase starter
<https://github.com/makerkit/nextjs-saas-starter-kit-lite>
- Or your own Next.js + Supabase setup

Styling:

- Starter kit defaults are acceptable
- The application should feel modern, clean, and visually coherent
- Basic attention should be given to spacing, typography, layout consistency, and usability
- Visual polish is not the primary evaluation criterion, but the UI should not appear unfinished or purely utilitarian

Deliverables

Please submit:

1. GitHub Repository

A GitHub repository containing the project.

2. README.md

The README should include:

- Setup instructions
- Supabase schema
- RLS summary
- High-level architecture
- ERD
- Key assumptions and trade-offs
- Known limitations
- What you would improve with more time

3. AI Engineering Blueprint

Please include a short document explaining how AI was used during development.

This should include:

- Which AI tools were used
- Development workflow
- Agentic system rulesets / instruction files
- Prompting or context strategy
- How AI-generated code was reviewed and validated

- Key decisions made by the candidate vs. the AI system

Examples of useful files:

- Cursor rules
- Claude Code instructions
- Codex workflow notes
- Project memory files
- Agent planning documents

This blueprint will be considered a strong positive signal.

4. Optional Live Deployment

A live deployment link is optional but encouraged.

Examples:

- Vercel
- Netlify
- Similar platforms

5. Optional Video Walkthrough

A 5–10 minute walkthrough is optional but encouraged.

It may cover:

- Architecture
- Data model
- RLS decisions
- AI workflow
- Trade-offs
- What you would improve next

Repository Access

To preserve fairness between candidates and avoid solution sharing, we strongly prefer a private GitHub repository.

Please grant access to: mehmetduvarci2011@gmail.com

If possible, please keep the repository private for at least 30 days after submission.

A public repository is also acceptable if private access is not feasible.

Deadline

Delivery deadline: **within 3 days** of receiving the case.

If you do not finish everything, please document unfinished parts clearly in the README and explain what you prioritized and why. This is fully acceptable.

Evaluation Criteria

We will evaluate based on:

- Architecture and structure
- Supabase usage and data security
- Team-based data modeling
- Product and user-flow thinking
- Code clarity and maintainability
- AI-native engineering workflow
- Communication and explanation quality
- Ownership mindset
- Trade-off awareness

We care more about how and why than feature completeness.

 Confidentiality & Ownership Notice

*This case is **for evaluation purposes only**.*

*All work you produce will remain **your intellectual property** and will **not be used, shared, or implemented** in any commercial or live project.*

*Your submission will be reviewed solely by our internal evaluation team and treated as **strictly confidential**.*